SEAM BAMODULEX

Giuseppe Silvi

May 6, 2026

author	Giuseppe Silvi
compile˙options	-lang cpp -fpga-mem-th 4 -ct 1 -es 1 -mcd 16 -mdd 1024 -mdy 33 -single -ftz 0
description	AmbiX → Tetrahedral decoder (Gerzon BA-module, AmbiX variant)
filename	bamodulex.dsp
license	GPL-3.0
name	SEAM BAMODULEX
vendor	SEAM
version	0.1.0

This document provides a mathematical description of the Faust program text stored in the `bamodulex.dsp` file. See the notice in Section **??** (page **??**) for details.

1 Mathematical definition of `process`

The *bamodulex* program evaluates the signal transformer denoted by `process`, which is mathematically defined as follows:

1. Output signals y_i for $i \in [1, 4]$ such that

$$y_1(t) = 0.5 \cdot (x_4(t) + x_3(t) + s_1(t))$$
$$y_2(t) = 0.5 \cdot (x_1(t) + x_4(t) - x_2(t) + x_3(t))$$
$$y_3(t) = 0.5 \cdot (x_1(t) + x_3(t) - x_2(t) + x_4(t))$$
$$y_4(t) = 0.5 \cdot (s_1(t) - x_3(t) + x_4(t))$$

2. Input signals x_i for $i \in [1, 4]$

3. Intermediate signal s_1 such that

$$s_1(t) = x_1(t) + x_2(t)$$

Figure 1: Block diagram of `process`

2 Block diagram of `process`

The block diagram of `process` is shown on Figure ?? (page ??).

3 Notice

- This document was generated using Faust version 2.85.5 on May 06, 2026.
- The value of a Faust program is the result of applying the signal transformer denoted by the expression to which the `process` identifier is bound to input signals, running at the f_S sampling frequency.
- Faust (*Functional Audio Stream*) is a functional programming language designed for synchronous real-time signal processing and synthesis applications. A Faust program is a set of bindings of identifiers to expressions that denote signal transformers. A signal s in S is a function mapping[1] times $t \in \mathbb{Z}$ to values $s(t) \in \mathbb{R}$, while a signal transformer is a function from S^n to S^m, where $n, m \in \mathbb{N}$. See the Faust manual for additional information (http://faust.grame.fr).
- Every mathematical formula derived from a Faust expression is assumed, in this document, to having been normalized (in an implementation-dependent manner) by the Faust compiler.
- A block diagram is a graphical representation of the Faust binding of an identifier I to an expression E; each graph is put in a box labeled by I. Subexpressions of E are recursively displayed as long as the whole picture fits in one page.
- The `bamodulex-mdoc/` directory may also include the following subdirectories:
  - `cpp/` for Faust compiled code;
  - `pdf/` which contains this document;
  - `src/` for all Faust sources used (even libraries);
  - `svg/` for block diagrams, encoded using the Scalable Vector Graphics format (http://www.w3.org/Graphics/SVG/);
  - `tex/` for the LaTeX source of this document.

[1] Faust assumes that $\forall\, s \in S, \forall\, t \in \mathbb{Z}, s(t) = 0$ when $t < 0$.

4 Faust code listings

This section provides the listings of the Faust code used to generate this document, including dependencies.

Listing 1: `bamodulex.dsp`

```
declare name    "SEAM BAMODULEX";
declare vendor  "SEAM";
declare version "0.1.0";
declare author  "Giuseppe Silvi";
declare license "GPL-3.0";
declare description "AmbiX  Tetrahedral decoder (Gerzon BA-module, AmbiX variant)";
//
// BAMODULEX    AmbiX-domain B-to-A module
//
// Decodes a first-order AmbiX (ACN/SN3D) signal to four loudspeakers
// placed at the vertices of a tetrahedron inscribed in a cube:
//
//   LFU    Left  Front Up
//   RFD    Right Front Down
//   RBU    Right Back Up
//   LBD    Left  Back  Down
//
// AmbiX channel ordering (ACN): a0 = W, a1 = Y, a2 = Z, a3 = X.
//
// The decoder matrix is orthogonal: each corner sums W and the three
// Cartesian components signed by the corner's position, scaled by 1/2.
//
//   LFU = (a0 + a1 + a2 + a3) / 2
//   RFD = (a0 - a1 - a2 + a3) / 2
//   RBU = (a0 - a1 + a2 - a3) / 2
//   LBD = (a0 + a1 - a2 - a3) / 2
//
// This is the AmbiX-domain analogue of Gerzon's ‘bamodule‘ (FuMa).
// Compensation shelving filters (Gerzon 1975) are intentionally
// omitted: the STONE tetrahedral loudspeaker amplifier handles the
// HF/LF correction downstream, so the plugin remains a pure matrix.
//
// References:
//   - Gerzon, "Ambisonics. Part two: Studio techniques" (1975)
//   - Malham, "Space in Music  Music in Space" (1998)
//   - Nachbar et al., "ambiX  A Suggested Ambisonics Format" (2011)
//
//


//
import("stdfaust.lib");
//
bamodulex(a0,a1,a2,a3) = lfu, rfd, rbu, lbd
with {
    lfu = (a0 + a1 + a2 + a3) / 2;
    rfd = (a0 - a1 - a2 + a3) / 2;
    rbu = (a0 - a1 + a2 - a3) / 2;
    lbd = (a0 + a1 - a2 - a3) / 2;
};
//
process = bamodulex;
```

Listing 2: `stdfaust.lib`

```
//################################## stdfaust.lib ##########################################
// The purpose of this library is to give access to all the Faust standard libraries
// through a series of environments.
//##########################################################################################

aa = library("aanl.lib");
sf = library("all.lib");
an = library("analyzers.lib");
ba = library("basics.lib");
co = library("compressors.lib");
db = library("debug.lib");
de = library("delays.lib");
dm = library("demos.lib");
dx = library("dx7.lib");
en = library("envelopes.lib");
fd = library("fds.lib");
fi = library("filters.lib");
ho = library("hoa.lib");
hy = library("hysteresis.lib");
it = library("interpolators.lib");
la = library("linearalgebra.lib");
ma = library("maths.lib");
mi = library("mi.lib");
mo = library("motion.lib");
ef = library("misceffects.lib");
no = library("noises.lib");
os = library("oscillators.lib");
pf = library("phaflangers.lib");
pl = library("platform.lib");
pm = library("physmodels.lib");
qu = library("quantizers.lib");
rm = library("reducemaps.lib");
re = library("reverbs.lib");
ro = library("routes.lib");
sp = library("spats.lib");
si = library("signals.lib");
so = library("soundfiles.lib");
sy = library("synths.lib");
ve = library("vaeffects.lib");
vl = library("version.lib");
wa = library("webaudio.lib");
wd = library("wdmodels.lib");
```